The dyadic machine

adic's model of computation is a machine whose memory is a complete binary tree: at grade n, the machine D_n owns a tree of depth n with one bit at each of its 2^n leaves, and nothing else. A fixed finite program drives k cursors ("heads") over this tree; the only motions are `up`, `down0`, `down1`, and the only data operations are reading or writing the bit under a head resting at a leaf. Every instruction costs 1. Space is not measured — the grade n **is** the space, by construction. One program runs uniformly at every grade; the tower $\{D_n\}$, never any single machine, is the object of the theory (AIP-2).

This exposition presents the machine and its first mechanized results: the cost model is **structural** — streaming cheapness and logarithmic random access are theorems about the tree's geometry, not assumptions about a memory controller.

1 Movement and cost

A run is a word of instructions; its cost is its length. Movement composes as a monoid action, and every cost statement below is a closed-form total over a concrete word (no potentials, no credits). Positions are bit-paths; the metric is the tree distance $d(a, b) = |a| + |b| - 2|\mathrm{lcp}(a, b)|$.

LEAN ⊢ `Adic.Dyadic.random_access` **Random access costs the depth**

Descending the path p from the root reaches the leaf addressed by p at cost exactly n — the depth. Access cost is the path length, definitionally.

```
theorem random_access {n : Nat} (p : Path n) :
  ∃ final,
    run (descend p) (rootHead n) = some final ∧
      leafPosition final = some p.toList ∧ cost (descend p) = n
```

axioms: propext hash: 90d693624072

LEAN ⊢ `Adic.Dyadic.random_access_optimal` **…and the depth is optimal**

No word reaching a leaf at depth n from the root costs less than n: the descent is not merely one way to get there, it is the cheapest.

```
theorem random_access_optimal {n : Nat} (p : Path n) (word : Word)
  (final : Head n) (hrun : run word (rootHead n) = some final)
  (hleaf : leafPosition final = some p.toList) : n ≤ cost word
```

axioms: Classical.choice, Quot.sound, propext hash: 384959b54737

LEAN ⊢ `Adic.Dyadic.movement_cost_lower_bound` **Cost dominates the metric**

Any successful run moving a head from position a to position b costs at least $d(a, b)$ —

```
theorem movement_cost_lower_bound {n : Nat} (word : Word)
  (source target : Head n) (hrun : run word source = some target) :
  treeDistance source target ≤ cost word
```

axioms: Classical.choice, Quot.sound, propext hash: a9dd1cc5c5b7

LEAN ⊢ `Adic.Dyadic.movement_cost_realizable` **…and the metric is realizable**

— and some word achieves exactly $d(a, b)$: up to the least common ancestor, down the other side. Cost, restricted to movement, **is** the tree metric.

```
theorem movement_cost_realizable {n : Nat} (source target : Head n) :
  ∃ word, run word source = some target ∧ cost word = treeDistance source target
```

axioms: Classical.choice, Quot.sound, propext hash: 441789e92c57

2 Streaming, or the odometer

Number the leaves left to right. Stepping from leaf i to leaf $i + 1$ costs $2(1 + v_2(i + 1))$ — the walk to the least common ancestor and back, governed by how far the binary carry propagates when incrementing i. A full scan telescopes: $\sum_{i<2^n}(1 + v_2(i)) \approx 2 \cdot 2^n$. Streaming is cheap on D_n for the same reason the odometer performs amortized $O(1)$ carries; the 2-adic valuation is the cost model seen from the address side.

LEAN ⊢ `Adic.Dyadic.streaming` **Streaming is amortized O(1) per leaf**

The Euler word of grade n succeeds from the root, returns to the root, costs at most $4 \cdot 2^n$, and its trace of visited leaves is **exactly** the 2^n leaves in left-to-right order — each leaf once, in order. (The exact cost is $4 \cdot 2^n - 4$.)

```
theorem streaming (n : Nat) :
  cost (euler n) ≤ 4 * 2 ^ n ∧
    run (euler n) (rootHead n) = some (rootHead n) ∧
      leafVisits (euler n) (rootHead n) = some (leftToRightLeaves n) ∧
        (leftToRightLeaves n).length = 2 ^ n
```

axioms: Quot.sound, propext hash: da2cf6d93911

3 Locality is compositional

Heads cannot observe one another — an instruction reads only the control state and the addressed head's local view. That makes disjointness **syntactic**: operations of heads confined to disjoint (prefix-incomparable) subtrees commute, and whole schedules can be interleaved freely without changing result or cost. This is the frame rule of the machine, and the ground the capability calculus will stand on.

LEAN ⊢ `Adic.Dyadic.disjoint_subtree_interleaving` **Interleaving invariance (theorem 4)**

For words confined to disjoint subtrees (shared memory, real writes), every interleaving that preserves per-head order yields the same final configuration at the same cost.

```
theorem disjoint_subtree_interleaving {k n : Nat}
  {left right first second : ActionWord k} (leftHead rightHead : Fin k)
  (leftRegion rightRegion : List Bool) (hheads : leftHead ≠ rightHead)
  (hdisjoint : PrefixIncomparable leftRegion rightRegion)
  (hleft : ConfinedWord left leftHead leftRegion)
  (hright : ConfinedWord right rightHead rightRegion)
  (hfirst : Interleaving left right first)
  (hsecond : Interleaving left right second) (config : ActionConfig n k)
  (hinvariant :
    RegionsInvariant config leftHead rightHead leftRegion rightRegion) :
  runActions first config = runActions second config ∧
    actionCost first = actionCost second
```

axioms: Classical.choice, Quot.sound, propext hash: c0fe61356058

4 Two programs

Zip is the first real program: two input heads stream two grade-n trees, a write head streams the grade-$(n + 1)$ interleaving. Its itinerary is **oblivious** — data-independent — which is itself a theorem: the schedule's shape (moves, read/write skeleton, hence cost) does not depend on the inputs; only the choice between `write0` and `write1` does.

LEAN ⊢ `Adic.Dyadic.zipWord_correct` **Zip is correct**

Running the zip schedule on inputs a, b (arbitrary initial output) succeeds and leaves memory holding a, b unchanged and the output subtree equal to their leaf-interleaving, heads back at start.

```
theorem zipWord_correct {n : Nat} (a b : Tree n) (output : Tree (n + 1)) :
  runActions (zipWord n a b) (zipStart a b output) =
    some (zipStart a b (interleave n a b))
```

axioms: Classical.choice, Quot.sound, propext hash: 14348ba5db73

LEAN ⊢ `Adic.Dyadic.zipWord_cost` **Zip is linear**

The schedule costs exactly $20 \cdot 2^n - 12$, independent of the data — amortized $O(1)$ per element, with the constant in plain sight.

```
theorem zipWord_cost {n : Nat} (a b : Tree n) :
  actionCost (zipWord n a b) = zipCost n
```

axioms: Quot.sound, propext hash: 197f534c6425

Copy is the second: a two-head streaming copy at exact cost $10 \cdot 2^n - 8$, whose doubling chain telescopes —

LEAN ⊢ `Adic.Dyadic.doublingCopyTotal_linear_bound` **Doubling telescopes**

The total cost of copying at every grade $0, \ldots, c$ (the doubling vector's copy chain) is at most $20 \cdot 2^c$: linear in the final size. This is the machine half of FVec's amortized $O(1)$ push.

```
theorem doublingCopyTotal_linear_bound (c : Nat) :
  doublingCopyTotal c ≤ 20 * 2 ^ c
```

axioms: Quot.sound, propext hash: 7bf1eddaf4cc

5 The honest logarithm

The RAM's $O(1)$ random access is the standard fiction; D prices what it hides. A word RAM of 2^s words of 2^v bits embeds in a grade-$(s + v)$ tree — a word is an aligned grade-v subtree, address arithmetic is path navigation — and:

LEAN ⊢ `Adic.Dyadic.ram_program_simulation` **The honest log**

Every RAM program of cost T runs on D at cost $\leq 10 \cdot T \cdot (s + 2^v)$ — $O(\log S + w)$ per RAM step. The logarithm was always there; the RAM model merely declines to charge for it.

```
theorem ram_program_simulation {s v : Nat} (program : RamProgram s)
  (config : RamConfig s v) :
  ∃ scratch',
    runActions (compileProgram program config) (simulatorStart config) =
        some (encodedConfig (runRam program config) scratch') ∧
      ScratchRepresents (runRam program config) scratch' ∧
        actionCost (compileProgram program config) ≤
          10 * ramCost program * (s + 2 ^ v)
```

axioms: Classical.choice, Quot.sound, propext hash: ae500d06e278

LEAN ⊢ `Adic.Dyadic.ram_action_simulation` **The reverse direction, per action**

Conversely, a fixed-register word RAM simulates each D action in at most a constant number of instructions (three suffice): addresses as padded words, moves as shifts. So the two models differ by exactly the logarithm — in one direction only.

```
theorem ram_action_simulation {k n : Nat} :
  ∃ c,
    ∀ (action : AddressedOp k) (config next : ActionConfig n k),
      actionStep action config = some next →
        ∃ target,
          runRegisterRam (actionRamProgram n action)
                (registerEncode action.head config) =
              some target ∧
            RegisterRep next target ∧
              registerRamCost (actionRamProgram n action) ≤ c
```

axioms: Quot.sound, propext hash: faf9a8c8b649

LEAN ⊢ `Adic.Dyadic.ram_program_reverse_simulation` **The reverse direction, whole programs**

And the per-action simulation lifts: every D action word of cost T compiles to a register-RAM run of cost $\leq 5T$ that tracks the full configuration. Target 3 of AIP-2 is now closed in both directions: D and the word RAM differ by exactly one logarithm, in exactly one direction — machine-checked.

```
theorem ram_program_reverse_simulation {k n : Nat} :
  ∃ c,
    ∀ (word : ActionWord k) (source final : ActionConfig n k)
      (start : RegisterRamConfig n k),
      RegisterRep source start →
        runActions word source = some final →
          ∃ target,
            runRegisterRam (compileActionWord n word) start = some target ∧
              RegisterRep final target ∧
                registerRamCost (compileActionWord n word) ≤ c * actionCost word
```

axioms: Quot.sound, propext hash: 32fa4d3c56aa

6 What is deliberately absent

No caches (weighted heads under a Kraft budget are the next layer — cache-v0 draft), no eviction or capacity cliffs (recorded gap: Kraft pricing is not cliff pricing), no allocation, no capabilities (they arrive as the calculus above, the machine stays algebra-free), and no completion of the tower: $\mathbb{N}$ and $\mathbb{Z}_2$ are its two limits, and adic works with the diagram, never the limit (Letter 2).